

NAME: K.Naga Venkata Siva

REGNO: 192425345

Question 1: Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

- **Analysis of Memory Layers:**
 - **L1 Cache (Core-Private):** Split into instruction ($L1\text{-I}$) and data ($L1\text{-D}$) caches, operating at CPU core clock speeds ($<1\text{ ns}$) with typical sizes of $32\text{--}64\text{ KB}$. It holds the immediate operational state and active execution loops.
 - **L2 Cache (Cluster-Private):** Slightly larger ($512\text{ KB--}1\text{ MB}$) with $3\text{--}5\text{ ns}$ access times; acts as an intermediate buffer to catch inner-loop L1 evictions.
 - **L3 Cache / System-Level Cache (Shared):** Shared across CPU, GPU, and NPU ($4\text{--}16\text{ MB}$; $10\text{--}20\text{ ns}$ latency); retains working sets across background applications.
 - **DRAM (LPDDR5/5X):** High-capacity main system memory with $100\text{--}200\text{ ns}$ latency, delivering $50\text{--}60\text{ GB/s}$ bandwidth.
- **Root Cause:** App switching forces a CPU context switch. The incoming application suffers cold misses across L1/L2 caches, stalling execution while data is refetched from high-latency LPDDR DRAM.
- **Redesign Solution:**
 - Implement an on-chip **System-Level Cache (SLC)** outside the CPU cluster to buffer recently switched background application states before writing back to DRAM.
 - Employ **victim caches** and dynamic cache-partitioning algorithms to reserve cache ways for foreground UI-thread critical paths.
 - Integrate predictive hardware prefetchers coupled with OS scheduler hints to preload the incoming app context into L2/L3 before the context switch finishes.

Question 2: Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

- **Cache Hierarchy vs. Virtual Memory Paging:**
 - **Cache Hierarchy:** Entirely hardware-managed (SRAM); operates on fixed cache lines (usually 64 bytes) with latencies ranging from 1 ns to 20 ns . Cache misses trigger memory bus access cycles.
 - **Virtual Memory Paging:** Managed cooperatively by the OS and Memory Management Unit (MMU); handles data in pages (4 KB , 2 MB , or 1 GB) using Translation Lookaside Buffers (TLB) and page tables. Page faults incur heavy latency penalties (microseconds to milliseconds) from disk or network storage fetches.
- **Root Cause:** Large database workloads with non-contiguous indexing and pointer chasing cause frequent TLB evictions, high page table walk overhead, and memory bus congestion.
- **Recommended Improvements:**
 - **Transparent Huge Pages (2 MB / 1 GB):** Reduce the depth of page table walks and expand effective TLB coverage.
 - **Compute Express Link (CXL):** Deploy CXL-attached memory pooling over PCIe Gen 5/6 to expand byte-addressable DRAM pools with minimal latency overhead.
 - **Persistent Memory / Storage-Class Memory (SCM):** Place active database indexes and write-ahead logs in byte-addressable non-volatile memory to avoid disk-bound serialization.

Question 3: Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

- **Evaluation of Memory Technologies:**
 - **SRAM:** Volatile, 6T configuration, ultra-fast access ($<1\text{ ns}$), high bandwidth; drawback is large physical footprint and high static leakage power.
 - **DRAM:** Volatile, 1T-1C cell, high density and cost-effective; drawback is higher latency ($50\text{--}80\text{ ns}$) and periodic refresh cycles that introduce nondeterministic latency spikes.
 - **MRAM (Magnetoresistive RAM):** Non-volatile (spin-torque transfer / MTJ), zero standby leakage, near-SRAM read speeds, high endurance, radiation-tolerant, and provides instant data retention upon power loss.
- **Proposed Memory Hierarchy Design:**
 - **Level 0/1 Scratchpad (SRAM):** Embedded directly inside vision accelerators and sensor-fusion DSPs for cycle-accurate, zero-latency processing of raw LiDAR and radar frames.

- **Level 2 Non-Volatile Mission-Critical Storage (MRAM):** Used for real-time localization tables, vehicle trajectory states, and instant crash/blackbox logging without requiring backup capacitors.
- **Level 3 High-Capacity Buffer (LPDDR5 / HBM):** High-bandwidth shared memory to store large neural network perception models and deep buffer video streams.

Question 4: AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

- **Comparison of Memory Technologies:**
 - **NAND Flash:** High storage density and non-volatile; however, block-level reads/writes, slow random access ($\sim 10^3$ range), and high active energy make direct execution inefficient.
 - **DRAM:** High data transfer rates; however, continuous capacitor refresh requirements cause significant idle/standby power leakage (> 50 mW).
 - **RRAM (Resistive RAM):** Non-volatile memristor technology, byte-addressable, high read speeds, low switching energy, and native support for analog In-Memory Computing.
- **Optimized Hierarchical Memory Structure:**
 - **Non-Volatile Model Storage (RRAM Array):** Store quantized neural network weights directly in an embedded RRAM macro, enabling instant wake-up without loading large weights over external buses.
 - **Compute-In-Memory (CiM) Core:** Perform vector-matrix multiplications directly inside the RRAM crossbar array in the analog/charge domain to eliminate data movement bottlenecks.
 - **Activation Buffer (On-chip SRAM):** Use a small SRAM buffer to store dynamic layer activations and intermediate feature maps between layers.

Question 5: High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

- **L3 Cache vs. DRAM Throughput Analysis:**
 - **L3 Cache:** Delivers massive local bandwidth (~ 100 TB/s) at low latency (~ 10 ns), but limited capacity (~ 32 MB) cannot hold modern 4K/8K scene textures and geometry datasets.

- **DRAM (GDDR6 / HBM):** High capacity (16–24 GB) with 400–900 GB/s throughput; during scene transitions, simultaneous CPU asset requests and GPU draw calls overwhelm memory channels, causing bus contention and frame stutter.
- **Memory Hierarchy Enhancements:**
 - **Unified Memory Architecture (UMA):** Share a single, high-bandwidth GDDR6/HBM memory pool between CPU and GPU with zero-copy addressing, eliminating CPU-to-GPU data duplication.
 - **Hardware Sampler Feedback & Direct SSD-to-Memory Streaming:** Stream only visible mipmap tiles from an NVMe SSD through a dedicated hardware decompression engine directly into DRAM/L3, bypassing CPU decompression overhead.
 - **Large On-Die Intermediate Cache (Infinity Cache / eDRAM):** Implement a 64–128 MB dedicated high-speed SRAM cache layer to absorb frame-buffer operations (MSAA, ray tracing bounding boxes, render targets) and free up DRAM bandwidth for asset streaming.